LA(1)LR(0)

임기정

Abstract

This note fixes the LR(0) characteristic automaton, defines the one-symbol lookahead set attached to each completed LR(0) item, and derives the usual Read/Follow digraph computation. The proofs are written so that the essential claims appear as inclusions, fixed-point equations, and derivation chains.

Contents

1 The LR(0) Automaton

Definition 1 (LRA(G)). Let $G \triangleq (N, T, P, S)$ be a context-free grammar. Define:

(i)
$$G' = (N', T', P', S') \triangleq (N \uplus \{S'\},\, T \uplus \{\$\},\, P \cup \{[S' \to S\$]\},\, S').$$

(ii)
$$V \triangleq N \uplus T, \qquad V' \triangleq N' \uplus T'.$$

(iii)
$$\beta \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma \quad \overset{\text{def}}{\Longleftrightarrow} \quad (\beta, \gamma) \in \{(\alpha A z, \alpha\omega z) \mid [A \to \omega] \in P',\ \alpha \in V'^*,\ z \in T'^*\}.$$

(iv)
$$I_{\mathrm{LR}(0)} \triangleq \{[A \to \alpha \cdot \beta] \mid [A \to \alpha\beta] \in P'\}.$$

(v) For $q \subseteq I_{\mathrm{LR}(0)}$,
$$\mathrm{Cl}(q) =_\mu q \cup \{[A \to \varepsilon \cdot \omega] \mid [A \to \omega] \in P',\ [B \to \beta \cdot A\gamma] \in \mathrm{Cl}(q)\}.$$

(vi)
$$q_0 \triangleq \mathrm{Cl}(\{[S' \to \varepsilon \cdot S\$]\}).$$

(vii) For $q \subseteq I_{\mathrm{LR}(0)}$ and $X \in V'$,
$$\mathrm{GOTO}(q, X) \triangleq \mathrm{Cl}(\{[A \to \alpha X \cdot \beta] \mid [A \to \alpha \cdot X\beta] \in q\}).$$

(viii)
$$Q \triangleq \mathrm{PT} \setminus \{\emptyset\}, \qquad \mathrm{PT} =_\mu \{q_0\} \cup \{\mathrm{GOTO}(q, X) \mid q \in \mathrm{PT},\ X \in V'\}.$$

(ix) The path relation is the least relation satisfying
$$\varepsilon : p \to q \iff p \in Q \wedge p = q, \qquad X\alpha : p \to q \iff p \in Q \wedge \alpha : \mathrm{GOTO}(p, X) \to q.$$

(x)
$$\mathbf{Config} \triangleq \{(\alpha : p \to q, z) \mid \alpha \in V'^*,\ p, q \in Q,\ z \in T'^*,\ \alpha : p \to q\}.$$

(xi)
$$\delta(q, X) \triangleq \mathrm{GOTO}(q, X) \qquad (q \in Q,\ X \in V').$$

(xii)
$$\mathbf{reduce}(q, t) \triangleq \{[A \to \omega] \mid [A \to \omega \cdot \varepsilon] \in q\} \qquad (q \in Q,\ t \in T').$$

(xiii) Let $\vdash \subseteq \mathbf{Config} \times \mathbf{Config}$ be generated by:
$$\frac{q'' = \delta(q', t)}{(\alpha : q \to q', tz) \vdash (\alpha t : q \to q'', z)}\ \text{Shift}$$
$$\frac{\alpha : q \to p \quad \omega : p \to q' \quad [A \to \omega] \in \mathbf{reduce}(q', t) \quad q'' = \delta(p, A)}{(\alpha\omega : q \to q', tz) \vdash (\alpha A : q \to q'', tz)}\ \text{Reduce}(A \to \omega)$$

(xiv)
$$q_f \triangleq \delta(\delta(q_0, S), \$).$$

(xv) The accepted language is defined as
$$L(\mathrm{LRA}(G)) \triangleq \{z \in T^* \mid (\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon)\}.$$

For $c = (\alpha : p \to q, u) \in \mathbf{Config}$, set $\Upsilon(c) \in V'^*$ as

$$\Upsilon(c) \triangleq \alpha u.$$

Lemma 2 (One-step yield invariant). *Each parser step reverses zero or one rightmost grammar step:*

$$c \vdash c' \implies \begin{cases} \Upsilon(c') = \Upsilon(c), & \text{if the step is Shift,} \\ \Upsilon(c') \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \Upsilon(c), & \text{if the step is Reduce.} \end{cases} \tag{1}$$

Proof. Indeed,

$$\begin{array}{rrcll} \text{Shift}: & \Upsilon(\alpha : q \to q', tz) & = & \alpha tz & = \Upsilon(\alpha t : q \to q'', z), \\ \text{Reduce}: & \Upsilon(\alpha A : q \to q'', tz) & = & \alpha A tz & \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha\omega tz = \Upsilon(\alpha\omega : q \to q', tz). \end{array}$$

□

Corollary 3 (Multi-step yield invariant). *For $c, c' \in \mathbf{Config}$,*

$$c \vdash^* c' \implies \Upsilon(c') \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \Upsilon(c). \tag{2}$$

Proof. Induct on the length of $c \vdash^* c'$ and use Lemma 2 at each parser step. □

Corollary 4 (LR(0) soundness).

$$L(\mathrm{LRA}(G)) \subseteq L(G).$$

Proof.

$$\begin{aligned} z \in L(\mathrm{LRA}(G)) &\implies (\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon) \\ &\overset{(2)}{\implies} S\$ \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} z\$ \\ &\implies S \Rightarrow^*_P z \\ &\implies z \in L(G). \end{aligned}$$

The third implication deletes the endmarker. Write the rightmost derivation witnessing $S\$ \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} z\$$ as

$$S\$ = \gamma_0 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_1 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \cdots \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_M = z\$.$$

Since S' occurs on no right-hand side of P', the only production mentioning S', namely $[S' \to S\$]$, is never applicable to a form derived from $S\$$; hence every step uses a production of $P \subseteq P'$. Moreover $\$ \notin V$ occurs on no right-hand side of P, and no production of P rewrites a terminal, so the rightmost symbol $\$$ of γ_0 is never touched:

$$\gamma_k = \delta_k\$, \qquad \delta_k \in V^* \quad (0 \le k \le M), \qquad \delta_0 = S, \quad \delta_M = z, \qquad \delta_k \Rightarrow_P \delta_{k+1}.$$

Stripping the common trailing $\$$ therefore yields $S = \delta_0 \Rightarrow_P \delta_1 \Rightarrow_P \cdots \Rightarrow_P \delta_M = z$, i.e. $S \Rightarrow^*_P z$. □

Fact 5 (Elimination of non-productive nonterminals). *Call $A \in N'$* productive *if $A \Rightarrow^*_{P'} w$ for some $w \in T'^*$, and* non-productive *otherwise. Removing every non-productive nonterminal together with all productions that mention one preserves the generated language. Hence, whenever $L(G) \neq \emptyset$, we may assume without loss of generality that every nonterminal of G' is productive; when $L(G) = \emptyset$ the identity $L(\mathrm{LRA}(G)) = L(G) = \emptyset$ already follows from soundness (Corollary 4).*

Proof. Extend productivity to strings: $\sigma \in V'^*$ is productive if $\sigma \Rightarrow^*_{P'} x$ for some $x \in T'^*$. A terminal derives only itself and a derivation of a string is the concatenation of derivations of its symbols, so

$$Y_1 \cdots Y_m \text{ productive} \iff \forall i.\ Y_i \text{ productive} \qquad (t \in T' \text{ productive}). \tag{3}$$

Fixed-point characterization. Let $\mathrm{Gen} \subseteq N'$ be the least fixed point

$$\mathrm{Gen} = \bigcup_{n\geq 0} \mathrm{Gen}_n, \quad \mathrm{Gen}_0 = \emptyset, \quad \mathrm{Gen}_{n+1} = \{A \in N' \mid \exists\, [A \to X_1 \cdots X_k] \in P'.\ \forall i.\ X_i \in T' \cup \mathrm{Gen}_n\}$$

(with $k = 0$, i.e. $[A \to \varepsilon] \in P'$, permitted). Then A is productive iff $A \in \mathrm{Gen}$:

$$\begin{aligned} A \in \mathrm{Gen}_{n+1} &\Longrightarrow [A \to X_1 \cdots X_k] \in P' \text{ with } X_i \in T' \cup \mathrm{Gen}_n \\ &\overset{\mathrm{IH}_n}{\Longrightarrow} A \Rightarrow_{P'} X_1 \cdots X_k \Rightarrow^*_{P'} x_1 \cdots x_k \in T'^*; \\ A \Rightarrow^{\ell}_{P'} w \in T'^* \ (\ell \geq 1) &\Longrightarrow A \Rightarrow_{P'} X_1 \cdots X_k \Rightarrow^{\ell-1}_{P'} w,\ X_i \Rightarrow^{\ell_i}_{P'} w_i,\ \ell_i < \ell \\ &\overset{\mathrm{IH}_\ell}{\Longrightarrow} X_i \in T' \cup \mathrm{Gen} \Longrightarrow A \in \mathrm{Gen}. \end{aligned}$$

Thus the non-productive nonterminals are exactly $N' \setminus \mathrm{Gen}$, a least fixed point.

Language preservation. Let $\widehat{G}$ delete $N' \setminus \mathrm{Gen}$ and every production mentioning it; since terminals are productive, its productions are

$$\widehat{P} \triangleq \{[A \to X_1 \cdots X_k] \in P' \mid A, X_1, \ldots, X_k \text{ all productive}\} \subseteq P'.$$

As $\widehat{P} \subseteq P'$, $S' \Rightarrow^*_{\widehat{P}} z$ implies $S' \Rightarrow^*_{P'} z$. For the converse, induct on the length ℓ of $\sigma \Rightarrow^{\ell}_{P'} x \in T'^*$:

$$\sigma \Rightarrow^{\ell}_{P'} x \in T'^* \implies \sigma \text{ productive } \wedge\ \sigma \Rightarrow^*_{\widehat{P}} x.$$

For $\ell = 0$, $\sigma = x \in T'^*$. For $\ell \geq 1$, split off the first step $\sigma = \sigma_1 C \sigma_2$, $[C \to \gamma] \in P'$, $\sigma_1, \sigma_2 \in V'^*$:

$$\sigma = \sigma_1 C \sigma_2 \Rightarrow_{P'} \sigma_1 \gamma \sigma_2 \Rightarrow^{\ell-1}_{P'} x \overset{\mathrm{IH}}{\Longrightarrow} \sigma_1 \gamma \sigma_2 \text{ productive } \wedge\ \sigma_1 \gamma \sigma_2 \Rightarrow^*_{\widehat{P}} x.$$

By (3), γ is productive, hence so is C (via $[C \to \gamma]$), so $[C \to \gamma] \in \widehat{P}$ and

$$\sigma = \sigma_1 C \sigma_2 \Rightarrow_{\widehat{P}} \sigma_1 \gamma \sigma_2 \Rightarrow^*_{\widehat{P}} x.$$

Taking $\sigma = S'$ gives $L(G') \subseteq L(\widehat{G})$, so $L(\widehat{G}) = L(G')$ and every nonterminal of $\widehat{G}$ is productive.

When $L(G) \neq \emptyset$, S is productive, hence S' is productive ($[S' \to S\$]$) and $\widehat{G}$ keeps the start symbol with the same language; replacing G' by $\widehat{G}$ loses no generality. When $L(G) = \emptyset$, $S \notin \mathrm{Gen}$ and Corollary 4 gives $L(\mathrm{LRA}(G)) \subseteq L(G) = \emptyset$ directly. □

The path relation factors through the state reached after any prefix: induction on $|\alpha|$ using Definition 1(ix) gives, for all $\alpha, \omega \in V'^*$ and $r \in Q$,

$$\alpha\omega : r \to q \iff \exists!\, p \in Q.\ \alpha : r \to p\ \wedge\ \omega : p \to q, \tag{4}$$

the intermediate state being the unique state obtained by following α from r, since each $\delta(\cdot, X) = \mathrm{GOTO}(\cdot, X)$ is single-valued.

Lemma 6 (LR(0) path invariants). *For $\eta \in V'^*$, the characteristic automaton satisfies*

$$(\exists q \in Q)\ \eta : q_0 \to q \iff \eta \textit{ is a viable prefix of } G', \tag{5}$$

$$\left.\begin{array}{c} \eta = \eta'\omega, \quad \eta' : q_0 \to p, \quad \omega : p \to q, \\ S' \underset{\mathbf{rm}}{\Longrightarrow}{}^*_{P'} \eta' A y \underset{\mathbf{rm}}{\Longrightarrow}{}_{P'} \eta' \omega y, \quad y = tz \in T'^* \end{array}\right\} \Longrightarrow [A \to \omega \cdot \varepsilon] \in q\ \wedge\ [A \to \omega] \in \mathbf{reduce}(q, t). \tag{6}$$

Proof. By Fact 5 we may assume every nonterminal of G' is productive: $C \Rightarrow^*_{P'} x$ for some $x \in T'^*$. Write $\sqsubseteq$ for the prefix order on V'^*. Call an item *valid* for $\eta \in V'^*$ when

$$[A \to \alpha \cdot \beta] \in \mathsf{Valid}(\eta) \overset{\text{def}}{\iff} \exists\, \delta \in V'^*, w \in T'^*.\ \eta = \delta\alpha\ \wedge\ S' \underset{\mathbf{rm}}{\Longrightarrow}{}^*_{P'} \delta A w \tag{7}$$

(then $[A \to \alpha\beta] \in P'$ and $w \in T'^*$ make $\delta A w \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \delta\alpha\beta w$ a rightmost step). Validity captures viability:

$$\mathsf{Valid}(\eta) \neq \emptyset \iff \eta \text{ is a viable prefix of } G', \tag{8}$$

since $[A \to \alpha \cdot \beta] \in \mathsf{Valid}(\eta)$ yields $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \delta A w \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \delta\alpha\beta w$ with $\eta = \delta\alpha \sqsubseteq \delta\alpha\beta$, while a viable prefix $\eta \sqsubseteq \delta\alpha\beta$ (from $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \delta A w \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \delta\alpha\beta w$) factors as $\eta = \delta\alpha\beta_1$, $\beta = \beta_1\beta_2$, giving $[A \to \alpha\beta_1 \cdot \beta_2] \in \mathsf{Valid}(\eta)$.

<u>Closure computes the valid items at a fixed η</u>. If $q \subseteq \mathsf{Valid}(\eta)$ contains every valid item with nonempty left part (and the seed $[S' \to \varepsilon \cdot S\$]$ when $\eta = \varepsilon$), then $\mathrm{Cl}(q) = \mathsf{Valid}(\eta)$. For $\subseteq$, the closure rule adds $[A \to \varepsilon \cdot \omega]$ only from some $[B \to \beta_1 \cdot A\beta_2] \in \mathrm{Cl}(q) \subseteq \mathsf{Valid}(\eta)$, and productivity $\beta_2 \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} x \in T'^*$ gives

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \delta B w \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \delta\beta_1 A\beta_2 w \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \delta\beta_1 A x w, \qquad \eta = \delta\beta_1,\ \ xw \in T'^* \implies [A \to \varepsilon \cdot \omega] \in \mathsf{Valid}(\eta).$$

For $\supseteq$, take $[A \to \varepsilon \cdot \omega] \in \mathsf{Valid}(\eta)$ with a witness $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \eta A w$ and induct on its length. Length 0 forces $\eta A w = S'$, i.e. $\eta = \varepsilon$, $A = S'$, the seed. Otherwise the step first introducing this occurrence of A reads

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \mu C \rho \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \mu\sigma_1 A \sigma_2 \rho, \qquad [C \to \sigma_1 A \sigma_2] \in P', \quad \eta = \mu\sigma_1, \quad \rho \in T'^*,$$

so $[C \to \sigma_1 \cdot A\sigma_2] \in \mathsf{Valid}(\eta)$ by the shorter witness $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \mu C \rho$. This item lies in q (if $\sigma_1 \neq \varepsilon$) or in $\mathrm{Cl}(q)$ (induction hypothesis, if $\sigma_1 = \varepsilon$); its dot precedes A, so the closure rule appends $[A \to \varepsilon \cdot \omega] \in \mathrm{Cl}(q)$.

<u>GOTO shifts the prefix</u>. By (7), for every $X \in V'$,

$$[A \to \alpha \cdot X\beta] \in \mathsf{Valid}(\eta) \iff [A \to \alpha X \cdot \beta] \in \mathsf{Valid}(\eta X) \qquad (\text{same } \delta, w;\ \eta = \delta\alpha \iff \eta X = \delta\alpha X),$$

and every nonempty-left-part valid item for ηX ends in X, hence has this form. These items are the GOTO-kernel; closing it gives

$$q = \mathsf{Valid}(\eta) \implies \mathrm{GOTO}(q, X) = \mathsf{Valid}(\eta X). \tag{9}$$

<u>Induction on $|\eta|$</u>. No valid item for ε has a nonempty left part, so $q_0 = \mathrm{Cl}(\{[S' \to \varepsilon \cdot S\$]\}) = \mathsf{Valid}(\varepsilon)$. Inductively, $\eta X : q_0 \to q'$ means $\eta : q_0 \to q$ with $q = \mathsf{Valid}(\eta)$ and $q' = \delta(q, X) = \mathrm{GOTO}(q, X) \overset{(9)}{=} \mathsf{Valid}(\eta X)$. Hence

$$\eta : q_0 \to q \implies q = \mathsf{Valid}(\eta). \tag{10}$$

Because every state on a path lies in $Q = \mathrm{PT} \setminus \{\emptyset\}$, and prefixes of a viable prefix are viable (so a run never blocks),

$$(\exists q \in Q)\, \eta : q_0 \to q \overset{(10)}{\iff} \mathsf{Valid}(\eta) \neq \emptyset \overset{(8)}{\iff} \eta \text{ is a viable prefix of } G',$$

which is (5).

<u>Handle invariant</u>. With $\eta = \eta'\omega$, $\eta' : q_0 \to p$, $\omega : p \to q$, and $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \eta' A y \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \eta'\omega y$, $y = tz \in T'^*$: (4) gives $\eta'\omega : q_0 \to q$, so $q = \mathsf{Valid}(\eta'\omega)$ by (10). Taking $\delta = \eta'$, $w = y$ in (7),

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \eta' A y, \quad y \in T'^* \implies [A \to \omega \cdot \varepsilon] \in \mathsf{Valid}(\eta'\omega) = q \overset{\text{Def. 1(xii)}}{\Longrightarrow} [A \to \omega] \in \mathbf{reduce}(q, t),$$

which is (6). □

Lemma 7 (LR(0) completeness).

$$L(G) \subseteq L(\mathrm{LRA}(G)).$$

Proof. Let $z \in L(G)$, so $S \Rightarrow_P^* z$. Choose a rightmost derivation in G':

$$S\$ = \gamma_0 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_1 \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \cdots \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \gamma_N = z\$. \tag{11}$$

Write its k-th step as

$$\gamma_k = \alpha_k A_k y_k \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha_k \omega_k y_k = \gamma_{k+1}, \qquad [A_k \to \omega_k] \in P', \quad y_k = t_k z_k \in T'^*. \tag{12}$$

By (6), for each k there exist $p_k, q_k \in Q$ such that

$$\alpha_k : q_0 \to p_k, \qquad \omega_k : p_k \to q_k, \qquad [A_k \to \omega_k \cdot \varepsilon] \in q_k, \qquad [A_k \to \omega_k] \in \mathbf{reduce}(q_k, t_k). \tag{13}$$

Reading (11) from right to left turns each step into one inverse reduction. By (13) and (4), $\alpha_k\omega_k : q_0 \to q_k$, so the Reduce$(A_k \to \omega_k)$ rule applies:

$$(\alpha_k\omega_k : q_0 \to q_k, y_k) \vdash (\alpha_k A_k : q_0 \to \delta(p_k, A_k), y_k) \qquad (k = N-1, \ldots, 0). \tag{14}$$

These reductions are interleaved with Shift blocks. Since $y_k \in T'^*$, the symbol A_k exposed at step k is the rightmost nonterminal of $\gamma_k = \alpha_k A_k y_k$; using the previous step $\gamma_k = \alpha_{k-1}\omega_{k-1}y_{k-1}$ with $y_{k-1} \in T'^*$ and cancelling the common terminal suffix,

$$\alpha_k A_k \, u_k = \alpha_{k-1}\omega_{k-1}, \qquad y_k = u_k \, y_{k-1}, \qquad u_k \in T'^* \qquad (1 \le k \le N-1). \tag{15}$$

By (4) the path $\alpha_{k-1}\omega_{k-1} : q_0 \to q_{k-1}$ factors through $\delta(p_k, A_k)$ after the prefix $\alpha_k A_k$, so the $|u_k|$ symbols of u_k are shifted one by one:

$$(\alpha_k A_k : q_0 \to \delta(p_k, A_k),\, u_k y_{k-1}) \ \vdash^{|u_k|} \ (\alpha_{k-1}\omega_{k-1} : q_0 \to q_{k-1},\, y_{k-1}).$$

At the bottom ($k = N-1$), the run starts by shifting all of $\alpha_{N-1}\omega_{N-1}$ out of $z\$ = \gamma_N$:

$$(\varepsilon : q_0 \to q_0, z\$) \ \vdash^{|\alpha_{N-1}\omega_{N-1}|} \ (\alpha_{N-1}\omega_{N-1} : q_0 \to q_{N-1}, y_{N-1}).$$

At the top ($k = 0$) we have $\gamma_0 = S\$$, whence $\alpha_0 = \varepsilon$, $A_0 = S$, $y_0 = \$$; (14) leaves the stack S reading $\$$, and one final Shift of $\$$ gives

$$(S : q_0 \to \delta(q_0, S), \$) \ \vdash \ (S\$: q_0 \to q_f, \varepsilon), \qquad q_f = \delta(\delta(q_0, S), \$).$$

Concatenating the Shift blocks with (14) for $k = N-1, \ldots, 0$ gives

$$(\varepsilon : q_0 \to q_0, z\$) \vdash^* (S\$: q_0 \to q_f, \varepsilon),$$

hence $z \in L(\mathrm{LRA}(G))$. □

Corollary 8 (Correctness of $\mathrm{LRA}(G)$).

$$L(G) = L(\mathrm{LRA}(G)), \qquad L(G) \triangleq \{z \in T^* \mid S \Rightarrow_P^* z\}.$$

Proof. Corollary 4 and Lemma 7 give

$$L(\mathrm{LRA}(G)) \subseteq L(G), \qquad L(G) \subseteq L(\mathrm{LRA}(G)).$$

□

2 Read and Follow Sets

Define $\mathbf{LA} : \{(q, [A \to \omega]) \mid q \in Q,\ [A \to \omega \cdot \varepsilon] \in q\} \longrightarrow \mathcal{P}(T')$ by

$$\mathbf{LA}(q, [A \to \omega]) \triangleq \left\{ t \in T' \;\middle|\; S' \underset{\mathbf{rm}}{\Longrightarrow}^{*}_{P'} \alpha A t z,\ \alpha\omega : q_0 \to q,\ \alpha \in V'^{*},\ z \in T'^{*} \right\}. \tag{16}$$

The remainder of this section computes $\mathbf{LA}(q, [A \to \omega])$ from the LR(0) automaton, as the least fixed point of a Read/Follow digraph.

Let

$$D \triangleq \{(p, A) \mid p \in Q,\ A \in N',\ \delta(p, A) \neq \emptyset\}.$$

For a relation R, write

$$R\,!\,x \triangleq \{y \mid (x, y) \in R\}.$$

Define $\mathbf{Read}, \mathbf{Follow} \subseteq D \times T'$ as the least relations generated by the rules

$$\frac{\delta(\delta(p, A), t) \neq \emptyset}{t \in \mathbf{Read}\,!\,(p, A)}\ \mathrm{DR} \qquad\qquad \frac{\delta(p, A) = r \quad C \Rightarrow^{*}_{P'} \varepsilon \quad (r, C) \in D}{\mathbf{Read}\,!\,(r, C) \subseteq \mathbf{Read}\,!\,(p, A)}\ \text{reads}$$

$$\frac{t \in \mathbf{Read}\,!\,(p, A)}{t \in \mathbf{Follow}\,!\,(p, A)}\ \text{Read} \qquad \frac{[B \to \beta \cdot A\gamma] \in p \quad \beta : p' \to p \quad \gamma \Rightarrow^{*}_{P'} \varepsilon \quad (p', B) \in D}{\mathbf{Follow}\,!\,(p', B) \subseteq \mathbf{Follow}\,!\,(p, A)}\ \text{includes}$$

Define the semantic read and follow sets on D:

$$\mathsf{Read}(p, A) \triangleq \{t \in T' \mid \exists r, s, \gamma.\ r = \delta(p, A) \wedge \gamma \Rightarrow^{*}_{P'} \varepsilon \wedge \gamma t : r \to s\}, \tag{17}$$

$$\mathsf{Follow}(p, A) \triangleq \left\{ t \in T' \;\middle|\; \exists \alpha, z.\ S' \underset{\mathbf{rm}}{\Longrightarrow}^{*}_{P'} \alpha A t z \wedge \alpha : q_0 \to p \wedge z \in T'^{*} \right\}. \tag{18}$$

For $(p, A) \in D$, set

$$\mathrm{DR}(p, A) \triangleq \{t \in T' \mid \delta(\delta(p, A), t) \neq \emptyset\}.$$

Let

$$\Phi_R(F)(p, A) = \mathrm{DR}(p, A) \cup \bigcup_{\substack{\delta(p,A)=r \\ C \Rightarrow^{*}_{P'} \varepsilon \\ (r,C) \in D}} F(r, C).$$

Lemma 9 (Semantic Read fixed point). *For $(p, A) \in D$,*

$$\mathbf{Read}\,!\,(p, A) = \mu\Phi_R(p, A) = \mathsf{Read}(p, A). \tag{19}$$

Proof. The two **Read**-rules say exactly that **Read**! is the least Φ_R-closed family, i.e. $\mathbf{Read}!(p, A) = \mu\Phi_R(p, A)$; since Φ_R is monotone on the finite lattice $D \to \mathcal{P}(T')$,

$$\mu\Phi_R = \bigcup_{m \geq 0} \Phi_R^m(\emptyset).$$

Write $(p, A) \leadsto_R (r, C)$ for the reads edge $r = \delta(p, A) \wedge C \Rightarrow^{*}_{P'} \varepsilon \wedge (r, C) \in D$. Because $\Phi_R(\emptyset)(p, A) = \mathrm{DR}(p, A)$, an induction on m unfolds the operator into reads-paths:

$$\Phi_R^{m+1}(\emptyset)(p, A) = \bigcup_{i=0}^{m} \bigcup_{(p,A) \leadsto_R^{i} (r,C)} \mathrm{DR}(r, C). \tag{20}$$

By (17), $t \in \mathsf{Read}(p, A)$ iff for some nullable $\gamma = C_1 \cdots C_n \in N'^{*}$,

$$r_0 = \delta(p, A), \qquad r_i = \delta(r_{i-1}, C_i)\ (1 \leq i \leq n), \qquad \delta(r_n, t) \neq \emptyset,$$

If $n = 0$, this says exactly that $t \in \mathrm{DR}(p, A)$. If $n > 0$, it is an n-edge reads-path ending at a DR-seed holding t:

$$(p, A) \rightsquigarrow_R (r_0, C_1) \rightsquigarrow_R \cdots \rightsquigarrow_R (r_{n-1}, C_n), \qquad t \in \mathrm{DR}(r_{n-1}, C_n) \quad (\delta(\delta(r_{n-1}, C_n), t) = \delta(r_n, t) \neq \emptyset).$$

In both cases (20) puts t in $\mu\Phi_R(p, A)$, and every member of the union (20) reads back as such a nullable δ-chain. Hence (19). □

Let

$$\Phi_F(F)(p, A) = \mathsf{Read}(p, A) \cup \bigcup_{\substack{[B \to \beta \cdot A\gamma] \in p \\ \beta : p' \to p \\ \gamma \Rightarrow^*_{P'} \varepsilon \\ (p', B) \in D}} F(p', B).$$

Lemma 10 (Semantic Follow fixed point)**.** *For* $(p, A) \in D$*,*

$$\mathbf{Follow}\,!\,(p, A) = \mu\Phi_F(p, A) = \mathsf{Follow}(p, A). \tag{21}$$

Proof. By Lemma 9, the two **Follow**-rules define the least fixed point of Φ_F. First Follow is closed under Φ_F. For the Read-seed, fix $t \in \mathsf{Read}(p, A)$. As $p \in Q$ is reachable from q_0 by construction of PT, pick α with $\alpha : q_0 \to p$; by (17) there is a nullable γ with $\gamma t : \delta(p, A) \to s$, so (4) yields $\alpha A \gamma t : q_0 \to s$, and (5) makes $\alpha A \gamma t$ a viable prefix, i.e. $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A \gamma t z$ for some $z \in T'^*$. Hence

$$\begin{aligned} t \in \mathsf{Read}(p, A) &\Longrightarrow \exists \alpha, \gamma, z.\ \alpha : q_0 \to p \wedge \gamma \Rightarrow^*_{P'} \varepsilon \wedge S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A \gamma t z \\ &\Longrightarrow S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A \gamma t z \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z \qquad (\gamma \Rightarrow^*_{P'} \varepsilon \text{ rewritten rightmost}) \\ &\Longrightarrow t \in \mathsf{Follow}(p, A). \end{aligned}$$

For an includes edge, since $t \in \mathsf{Follow}(p', B), [B \to \beta \cdot A\gamma] \in p, \beta : p' \to p,$ and $\gamma \Rightarrow^*_{P'} \varepsilon$,

$$\begin{aligned} S' &\underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' B t z \\ &\underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha' \beta A \gamma t z \\ &\underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' \beta A t z, \end{aligned}$$

with $\alpha'\beta : q_0 \to p$; hence

$$\mathsf{Follow}(p', B) \subseteq \mathsf{Follow}(p, A).$$

Conversely, let $t \in \mathsf{Follow}(p, A)$, witnessed by

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z, \qquad \alpha : q_0 \to p.$$

The production $B \to \beta A \gamma$ that created this occurrence of A gives a factorization

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' B t' z' \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha' \beta A \gamma t' z' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z, \qquad \alpha = \alpha'\beta, \qquad t'z' \in T'^*,$$

in which the last segment rewrites only the suffix, $\gamma t' z' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} tz$, leaving this A untouched. Splitting on whether γ is nullable makes exactly one alternative hold:

$$\begin{cases} \gamma = \gamma_0 X \gamma_1, \quad \gamma_0 \Rightarrow^*_{P'} \varepsilon, \quad X \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} t\chi, & t \in \mathsf{Read}(p, A), \\ \gamma \Rightarrow^*_{P'} \varepsilon, \quad t = t', & t \in \mathsf{Follow}(p', B) \text{ and } (p, A) \rightsquigarrow_{\text{includes}} (p', B). \end{cases}$$

In the first case γ_0 is read off $\delta(p, A)$ by a nullable δ-chain and $t \in \mathrm{FIRST}(X)$ gives $\delta(\cdot, t) \neq \emptyset$, so $\gamma_0 t : \delta(p, A) \to s$ realizes (17). In the second, $\gamma \Rightarrow^*_{P'} \varepsilon$ forces $t = t'$, and

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' B t z, \quad \alpha' : q_0 \to p' \ ((4) \text{ on } \alpha = \alpha'\beta) \implies t \in \mathsf{Follow}(p', B),$$

the includes edge holding by $[B \to \beta \cdot A\gamma] \in p$, $\beta : p' \to p$, $\gamma \Rightarrow^*_{P'} \varepsilon$. Its witnessing derivation $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' Btz$ is strictly shorter, so induction on the length gives

$$\mathsf{Follow}(p, A) \subseteq \mu\Phi_F(p, A).$$

Together with closure, this proves (21). $\square$

Remark (toward a Coq formalization of the converse)**.** Every part of Lemma 10 except the converse inclusion transcribes to a proof assistant essentially verbatim: the closure direction exhibits explicit derivations and paths, so its existential witnesses are exactly the objects just constructed. The converse is the single step that does not, and it carries most of the formalization cost of this section.

The obstruction is the phrase "the production $B \to \beta A\gamma$ that created this occurrence of A", together with the factorization

$$S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha' Bt'z' \underset{\mathbf{rm}}{\Longrightarrow}_{P'} \alpha'\beta A\gamma t'z' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha Atz$$

that must leave *this* A untouched. On paper the occurrence is read off the displayed string; once formalized, a right-sentential form is a bare list of symbols in which A may occur many times, and $\underset{\mathbf{rm}}{\Longrightarrow}_{P'}$ records nothing about which occurrence is meant. The tracked occurrence must be promoted to first-class data, for which two encodings suffice.

1. *Marked derivations.* Refine $\underset{\mathbf{rm}}{\Longrightarrow}_{P'}$ to a relation `marked_derives` λ A ρ on a sentential form presented as $\lambda\, A\, \rho$ around a distinguished A, with one constructor per rewrite position: strictly inside the left context λ, strictly inside the right context ρ, or the introducing step $B \to \beta A\gamma$ that first plants the mark. The converse becomes structural induction on this relation; the right-context constructors yield the Read case, and the introducing constructor yields the includes edge together with the strictly shorter derivation that the induction consumes.

2. *Zipper sentential forms.* Represent the right-sentential form as a zipper (λ, A, ρ) — a left list, the focused symbol, a right list — and case-analyse each $\underset{\mathbf{rm}}{\Longrightarrow}_{P'}$ step by where it fires relative to the focus: strictly left, strictly right, or at the focused symbol. Then "leaves this A untouched" is precisely the side condition that the step fires off the focus, and the same trichotomy reappears as the three rewrite positions.

Under either encoding the informal split on whether γ is nullable (Read seed versus includes edge) becomes a decidable case analysis on the focused step, and the well-founded induction runs on the length of the marked or zippered derivation rather than on an unmarked $S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \cdots$. By contrast Lemma 9, Corollary 11, and Theorem 12 are first-order least-fixed-point manipulations over finite sets and need no such occurrence tracking.

Corollary 11 (Lookahead by Follow)**.** *Whenever $q \in Q$ and $[A \to \omega \cdot \varepsilon] \in q$,*

$$\mathbf{LA}(q, [A \to \omega]) = \{t \in T' \mid \exists p \in Q.\ \omega : p \to q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathbf{Follow}\,!\,(p, A)\}. \tag{22}$$

Proof. For $q \in Q$ and $[A \to \omega \cdot \varepsilon] \in q$,

$$\begin{aligned}
t \in \mathbf{LA}(q, [A \to \omega]) &\iff \exists \alpha, z.\ S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha Atz \wedge \alpha\omega : q_0 \to q \\
&\overset{(4)}{\iff} \exists p, \alpha, z.\ S' \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha Atz \wedge \alpha : q_0 \to p \wedge \omega : p \to q \\
&\overset{(18)}{\iff} \exists p.\ \omega : p \to q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathsf{Follow}(p, A) \\
&\overset{(21)}{\iff} \exists p.\ \omega : p \to q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathbf{Follow}\,!\,(p, A).
\end{aligned}$$

The side condition $\delta(p, A) \neq \emptyset$ is automatic for $A \neq S'$: tracing $[A \to \omega \cdot \varepsilon] \in q$ backward along $\omega : p \to q$ gives $[A \to \varepsilon \cdot \omega] \in p$, which is introduced by an item with the dot before A. For $A = S'$, both sides are empty because S' occurs on no right-hand side and the parser does not reduce $[S' \to S\$]$. This is (22). $\square$

Theorem 12 (Digraph computation of lookahead). *The least relations* $\mathbf{Read}, \mathbf{Follow}$ *generated above compute every lookahead set by the formula* (22)*.*

Proof. The Read and Follow fixed-point claims are Lemmas 9 and 10; applying them to the semantic definition of $\mathbf{LA}$ gives Corollary 11, which is the displayed formula of the theorem. ☐

3 Lookahead Restriction

Recall the lookahead set $\mathbf{LA}$ of (16). Replace the LR(0) reduction function by

$$\mathbf{reduce}_{\mathbf{LA}}(q,t) \triangleq \{[A \to \omega] \mid [A \to \omega \cdot \varepsilon] \in q,\ t \in \mathbf{LA}(q,[A \to \omega])\}. \tag{23}$$

Let $\vdash_{\mathbf{LA}}$ be the transition relation obtained from $\vdash$ by replacing $\mathbf{reduce}$ with $\mathbf{reduce}_{\mathbf{LA}}$, and let $L(\vdash_{\mathbf{LA}})$ be its accepted language.

Lemma 13 (Guarded soundness).

$$L(\vdash_{\mathbf{LA}}) \subseteq L(G).$$

Proof. From (23),

$$\mathbf{reduce}_{\mathbf{LA}}(q,t) \subseteq \mathbf{reduce}(q,t) \quad \Longrightarrow \quad \vdash_{\mathbf{LA}} \subseteq \vdash \quad \Longrightarrow \quad {\vdash_{\mathbf{LA}}}^* \subseteq \vdash^*.$$

Therefore

$$L(\vdash_{\mathbf{LA}}) \subseteq L(\mathrm{LRA}(G)) = L(G) \qquad \text{by Corollary 8.} \tag{24}$$

☐

Lemma 14 (Guarded completeness).

$$L(G) \subseteq L(\vdash_{\mathbf{LA}}).$$

Proof. Let $z \in L(G)$. In the accepting computation constructed in Lemma 7, every Reduce step has the form

$$(\alpha\omega : q_0 \to q, tz') \vdash (\alpha A : q_0 \to \delta(p,A), tz'),$$

where

$$S' \underset{\mathbf{rm}}{\Longrightarrow}_{P'} S\$ \underset{\mathbf{rm}}{\Longrightarrow}^*_{P'} \alpha A t z', \qquad \alpha\omega : q_0 \to q, \qquad [A \to \omega \cdot \varepsilon] \in q.$$

By (16),

$$t \in \mathbf{LA}(q,[A \to \omega]) \quad \Longleftrightarrow \quad [A \to \omega] \in \mathbf{reduce}_{\mathbf{LA}}(q,t).$$

Thus every Reduce step used in that accepting LR(0) computation is also legal for $\vdash_{\mathbf{LA}}$; Shift steps are unchanged. Hence

$$L(G) \subseteq L(\vdash_{\mathbf{LA}}).$$

☐

Corollary 15 (Language preservation).

$$L(\vdash_{\mathbf{LA}}) = L(G).$$

Proof. Combine Lemmas 13 and 14. ☐

Lemma 16 (Conflict-free table condition). *If the resulting table has no shift/reduce or reduce/reduce conflict, its action is single-valued under one-symbol lookahead, hence it is an* LALR(1) *table.*

Proof. At (q,t), the possible actions are

$$\delta(q,t) \quad \text{when this shift target is nonempty,} \qquad \mathbf{reduce}_{\mathbf{LA}}(q,t).$$

Absence of shift/reduce and reduce/reduce conflicts means that this action is single-valued under one-symbol lookahead, which is precisely the LALR(1) table condition. ☐

Theorem 17 (Lookahead-guarded reductions)**.** *If the guarded table is conflict-free, it is an* LALR(1) *parser accepting* $L(G)$*.*

Proof. Corollary 15 gives acceptance of $L(G)$, and Lemma 16 gives the LALR(1) table condition under the stated absence of conflicts. □

A Digraph Algorithm and Implementation Specification

This appendix gives an executable specification for computing the sets used in Theorem 12. The construction is the usual digraph view of LALR(1) lookahead computation: direct seed sets are placed on nodes, dependency edges describe set inclusions, and the least fixed point is obtained by collapsing strongly connected components and propagating set unions along the resulting DAG [1, 2, 3].

A.1 General Digraph Fixed-Point Problem

Let X be a finite set of nodes, let $E \subseteq X \times X$ be a directed edge relation, and let $\mathrm{Seed} : X \to \mathcal{P}(T')$. We use the edge orientation

$$x \rightsquigarrow y \qquad \text{to mean} \qquad F(y) \subseteq F(x).$$

Thus the desired solution is the least function $F : X \to \mathcal{P}(T')$ satisfying

$$F(x) = \mathrm{Seed}(x) \cup \bigcup_{x \rightsquigarrow y} F(y). \tag{25}$$

Algorithm DIGRAPH(X, E, Seed)**.**

1. Compute the strongly connected components of (X, E), for example by Tarjan's depth-first-search algorithm.
2. Build the condensation DAG $\mathcal{C}$. Its nodes are SCCs, and $C \rightsquigarrow D$ is an edge when some $x \in C$ and $y \in D$ satisfy $x \rightsquigarrow y$, with $C \neq D$.
3. Initialize, for every SCC C,
$$\mathrm{Val}(C) \leftarrow \bigcup_{x \in C} \mathrm{Seed}(x).$$
4. Process the SCCs in reverse topological order with respect to $\rightsquigarrow$, so every successor D of C has already been processed, and put
$$\mathrm{Val}(C) \leftarrow \mathrm{Val}(C) \cup \bigcup_{C \rightsquigarrow D} \mathrm{Val}(D).$$
5. Return $F(x) \triangleq \mathrm{Val}(C_x)$, where C_x is the SCC containing x.

Correctness condition. For every SCC C, after step 4,

$$\mathrm{Val}(C) = \bigcup \{\mathrm{Seed}(y) \mid \text{some } x \in C \text{ reaches } y \text{ in } (X, E)\}.$$

Therefore the returned F is exactly the least solution of (25). This is the only property of the graph algorithm needed by Theorem 12.

Complexity. Let $b = \lceil |T'|/w \rceil$, where w is the machine word size used for bitsets. SCC construction is $O(|X| + |E|)$, and set propagation is

$$O((|X| + |E|)b).$$

A simple work-list implementation that repeatedly applies

$$F(x) := F(x) \cup F(y) \qquad (x \rightsquigarrow y)$$

is also correct, but the SCC version avoids repeated propagation inside cycles.

A.2 Instance for Read

First compute nullable nonterminals by the least fixed point

$$\mathrm{Null}(A) \iff \exists [A \to X_1 \cdots X_k] \in P' \text{ such that every } X_i \text{ is nullable},$$

where terminals are never nullable and $k = 0$ is allowed. Extend this to strings by

$$\mathrm{Null}(X_1 \cdots X_k) \iff \bigwedge_{i=1}^{k} \mathrm{Null}(X_i).$$

For $x = (p, A) \in D$, define the direct-read seed

$$\mathrm{DR}(p, A) \triangleq \{t \in T' \mid \delta(\delta(p, A), t) \neq \emptyset\}.$$

Define $E_R \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_R (r, C) \iff \delta(p, A) = r \wedge C \in N' \wedge \mathrm{Null}(C) \wedge (r, C) \in D.$$

If $(r, C) \notin D$, then $\textbf{Read}\,!\,(r, C) = \emptyset$, so no edge is needed. Compute

$$\textbf{Read}\,!\,(p, A) \triangleq \mathrm{DIGRAPH}(D, E_R, \mathrm{DR})(p, A).$$

A.3 Instance for Follow

The seed for follow propagation is the already computed read set:

$$\mathrm{Seed}_F(p, A) \triangleq \textbf{Read}\,!\,(p, A).$$

Define $E_I \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_I (p', B) \iff [B \to \beta \cdot A\gamma] \in p \wedge \beta : p' \to p \wedge \mathrm{Null}(\gamma) \wedge (p', B) \in D.$$

The orientation means

$$\textbf{Follow}\,!\,(p', B) \subseteq \textbf{Follow}\,!\,(p, A).$$

Compute

$$\textbf{Follow}\,!\,(p, A) \triangleq \mathrm{DIGRAPH}(D, E_I, \mathrm{Seed}_F)(p, A).$$

A.4 Lookback Relation and Lookahead Construction

For every completed item $[A \to \omega \cdot \varepsilon] \in q$, define

$$\mathrm{LB}(q, A \to \omega) \triangleq \{(p, A) \in D \mid \omega : p \to q\}.$$

Then

$$\textbf{LA}(q, [A \to \omega]) \triangleq \bigcup_{(p,A) \in \mathrm{LB}(q, A \to \omega)} \textbf{Follow}\,!\,(p, A),$$

which is (22) written as an implementation step.

Implementation note. The query $\omega : p \to q$ can be implemented by following δ-edges from p along ω. The query $\beta : p' \to p$ in E_I can be implemented by caching reverse-path queries keyed by (β, p), or by storing predecessor edges during construction of the LR(0) automaton.

A.5 Parser-Table and Conflict Specification

After **LA** has been computed, construct actions as follows.

1. If $\delta(q, t) \neq \emptyset$ for $t \in T'$, add the shift action
$$q \xrightarrow{t} \delta(q, t).$$
2. For each $[A \to \omega \cdot \varepsilon] \in q$ and each $t \in \mathbf{LA}(q, [A \to \omega])$, add the reduce action $A \to \omega$ under lookahead t.
3. Report a shift/reduce conflict when both a shift action and a reduce action are present for the same pair (q, t).
4. Report a reduce/reduce conflict when two distinct reduce productions are present for the same pair (q, t).

The accepting condition remains the one in Definition 1: the parser accepts when it reaches
$$(S\$: q_0 \to q_f, \varepsilon).$$
Equivalently, in a conventional ACTION/GOTO table, q_f may be treated as the accepting state after the input marker has been consumed.

A.6 Summary Specification

The complete computation required by Theorem 12 is:

1. Build G', Q, and δ as in Definition 1.
2. Compute nullable nonterminals and nullable strings.
3. Build D, DR, and E_R; compute **Read** by DIGRAPH(D, E_R, DR).
4. Build E_I; compute **Follow** by DIGRAPH$(D, E_I, \mathbf{Read})$.
5. Build LB and compute each $\mathbf{LA}(q, [A \to \omega])$ by unioning the appropriate **Follow**-sets.
6. Construct the guarded reduce actions and check conflicts.

The postcondition is that the computed **Read**, **Follow**, and **LA** are the least sets satisfying Theorem 12, and hence the lookahead-restricted parser of Theorem 17 uses exactly the LALR(1) lookahead sets determined by the LR(0) automaton.

B Fuel-Free Parser Execution

The table specification above is relational. An executable parser can be obtained without adding a fuel argument exactly when no reachable reduce-only phase can run forever. This section states that condition using the actual parser stacks, not a finite control-only over-approximation.

Definition 18 (Run stacks and reachable phase heads)**.** A run stack is a viable stack path starting at the initial LR state:
$$\mathbf{Stack} \triangleq \{\alpha : q_0 \to q \mid \alpha \in V'^*,\ q \in Q,\ \alpha : q_0 \to q\}.$$

The corresponding run configurations are

$$\mathbf{Config}_{\text{run}} \triangleq \{(s,u) \mid s \in \mathbf{Stack},\ u \in T'^*\}.$$

For an input word $w \in T^*$, put

$$c_0(w) \triangleq (\varepsilon : q_0 \to q_0, w\$).$$

A run configuration is reachable when

$$\mathsf{Reach}(c) \quad\Longleftrightarrow\quad \exists w \in T^*.\ c_0(w) \vdash^*_{\mathbf{LA}} c.$$

For $s \in \mathbf{Stack}$ and $t \in T'$, say that (s,t) is a reachable reduce-phase head when

$$\mathsf{Phase}(s,t) \quad\Longleftrightarrow\quad \exists u \in T'^*.\ \mathsf{Reach}(s,tu).$$

Definition 19 (Exact reduce-phase relation)**.** For each lookahead $t \in T'$, define a relation $\Rightarrow_t \subseteq \mathbf{Stack} \times \mathbf{Stack}$ by

$$s \Rightarrow_t s'$$

iff there exist $\alpha \in V'^*$, $A \in N'$, $\omega \in V'^*$, and states $p, q, q' \in Q$ such that

$$s = (\alpha\omega : q_0 \to q), \qquad s' = (\alpha A : q_0 \to q'),$$

$$\alpha : q_0 \to p, \qquad \omega : p \to q, \qquad [A \to \omega] \in \mathbf{reduce_{LA}}(q,t), \qquad q' = \delta(p,A).$$

Thus $s \Rightarrow_t s'$ records one actual guarded Reduce step with lookahead t, including the stack prefix that is exposed by popping the handle.

Lemma 20 (Exactness of reduce phases)**.** *For $s, s' \in \mathbf{Stack}$, $t \in T'$, and $u \in T'^*$, the following are equivalent:*

$$(s,tu) \vdash_{\mathbf{LA}} (s',tu) \quad \textit{by a Reduce step}$$

and

$$s \Rightarrow_t s'.$$

Consequently $\Rightarrow_t$ is neither forgetting the pop predecessor nor combining incompatible stack contexts.

Proof. Expanding the guarded Reduce rule, a Reduce step from (s,tu) to (s',tu) has the form

$$s = (\alpha\omega : q_0 \to q), \qquad s' = (\alpha A : q_0 \to q'),$$

with witnesses $p \in Q$, $A \in N'$, and $\omega \in V'^*$ satisfying

$$\alpha : q_0 \to p, \qquad \omega : p \to q, \qquad [A \to \omega] \in \mathbf{reduce_{LA}}(q,t), \qquad q' = \delta(p,A).$$

These are exactly the witnesses required by Definition 19. The converse is the same argument read backwards. □

Definition 21 (Exact reduce-phase termination)**.** The guarded table is *phase-terminating* when there are no $t \in T'$ and no infinite sequence of stacks

$$s_0 \Rightarrow_t s_1 \Rightarrow_t s_2 \Rightarrow_t \cdots$$

with $\mathsf{Phase}(s_0, t)$.

Lemma 22 (Finite branching of exact reductions)**.** *For each $s \in \mathbf{Stack}$ and $t \in T'$, the set*

$$\{s' \in \mathbf{Stack} \mid s \Rightarrow_t s'\}$$

is finite. If the guarded table is conflict-free, this set has at most one element.

Proof. Write $s = \beta : q_0 \to q$. A successor of s under $\Rightarrow_t$ must be produced by some completed item $[A \to \omega] \in \mathbf{reduce_{LA}}(q, t)$. This set of items is finite. For a fixed $A \to \omega$, the equality $\beta = \alpha\omega$ determines at most one prefix α, and the deterministic LR path from q_0 along α determines at most one exposed state p. Then the successor state is forced to be $\delta(p, A)$. Hence only finitely many successors can occur. If the table is conflict-free, at most one Reduce action is available at the pair (q, t), so there is at most one successor. □

Lemma 23 (Heights for finitely branching terminating relations)**.** *Let $R \subseteq X \times X$ be finitely branching. For $x \in X$, suppose that no infinite chain*

$$x = x_0 \; R \; x_1 \; R \; x_2 \; R \cdots$$

starts at x. Then

$$H_R(x) \triangleq \max\left\{ n \in \mathbb{N} \;\middle|\; \exists x_0, \ldots, x_n \in X.\; x_0 = x \wedge \bigwedge_{0 \le i < n} x_i \; R \; x_{i+1} \right\}$$

is well-defined. Moreover, if $x \; R \; y$, then

$$H_R(y) < H_R(x).$$

Conversely, any map $H : X \to \mathbb{N}$ that strictly decreases along R excludes infinite R-chains.

Proof. Let S_x be the set of lengths appearing in the displayed maximum. It is nonempty because $0 \in S_x$. We first show that S_x is bounded. Suppose, toward a contradiction, that S_x is unbounded. Since R is finitely branching, x has only finitely many successors. If every successor y of x had bounded length set S_y, then S_x would be bounded by

$$\max(\{0\} \cup \{1 + \max S_y \mid x \; R \; y\}),$$

a contradiction. Hence some successor x_1 has unbounded S_{x_1}. Repeating the same argument constructs an infinite chain

$$x = x_0 \; R \; x_1 \; R \; x_2 \; R \cdots,$$

contrary to the hypothesis. Therefore S_x is bounded; as a nonempty bounded subset of $\mathbb{N}$, it has a maximum, so $H_R(x)$ is well-defined.

If $x \; R \; y$, every path of length $H_R(y)$ from y can be prefixed by the edge $x \; R \; y$, giving a path of length $H_R(y) + 1$ from x. Thus $H_R(x) \ge H_R(y) + 1$, and so $H_R(y) < H_R(x)$.

Finally, if $H : X \to \mathbb{N}$ strictly decreases along R, an infinite R-chain would give an infinite strictly descending sequence in $\mathbb{N}$, which is impossible. □

Definition 24 (Exact termination certificate)**.** For a conflict-free guarded table, a termination certificate is a rank function

$$\rho : \mathbf{Config}_{\text{run}} \to \mathbb{N}$$

such that every reachable Shift or Reduce step strictly decreases

$$\mu(c) \triangleq (|\text{input}(c)|, \rho(c)) \in \mathbb{N} \times \mathbb{N}$$

in lexicographic order. Concretely, whenever $\mathsf{Reach}(c)$ and $c \vdash_{\mathbf{LA}} c'$ is a Shift or Reduce step, one has

$$\mu(c') <_{\text{lex}} \mu(c).$$

Accept and Error configurations make no recursive call.

Definition 25 (Recursive-call relation)**.** Let $\mathsf{Call} \subseteq \mathbf{Config}_{\text{run}} \times \mathbf{Config}_{\text{run}}$ be the relation

$$c' \; \mathsf{Call} \; c$$

iff $\mathsf{Reach}(c)$, $c \vdash_{\mathbf{LA}} c'$, and that one parser step is either Shift or Reduce. Accept and Error configurations have no Call successors.

Lemma 26 (Certificate gives accessibility)**.** *If a termination certificate ρ exists, then every reachable run configuration c satisfies*

$$\mathrm{Acc}_{\mathsf{Call}}(c).$$

Equivalently, the recursive parser may make recursive calls only to Call*-smaller configurations.*

Proof. Put $\mu(c) = (|\mathrm{input}(c)|, \rho(c))$. By Definition 24, every Call-edge c' Call c satisfies

$$\mu(c') <_{\mathrm{lex}} \mu(c).$$

The lexicographic order on $\mathbb{N} \times \mathbb{N}$ is well-founded, and well-foundedness is preserved by inverse image along μ. Hence the measure order

$$c' \prec_\rho c \quad \Longleftrightarrow \quad \mu(c') <_{\mathrm{lex}} \mu(c)$$

is well-founded on run configurations. Since Call is a subrelation of $\prec_\rho$ on reachable configurations, every reachable configuration is accessible for Call. ☐

Definition 27 (Canonical exact rank)**.** Assume the guarded table is phase-terminating. For $s \in \mathbf{Stack}$ and $t \in T'$, define

$$H_t(s) \triangleq \begin{cases} \max\left\{ n \in \mathbb{N} \;\middle|\; \exists s_0, \ldots, s_n \in \mathbf{Stack}.\ s_0 = s \wedge \bigwedge\limits_{0 \le i < n} s_i \Rightarrow_t s_{i+1} \right\}, & \text{if } \mathsf{Phase}(s,t), \\ 0, & \text{otherwise.} \end{cases}$$

For a run configuration $c = (s, u)$, put

$$\rho_{\mathrm{ex}}(c) \triangleq \begin{cases} H_t(s), & u = tu' \text{ for some } t \in T', u' \in T'^*, \\ 0, & u = \varepsilon. \end{cases}$$

Theorem 28 (Sound and complete fuel-free criterion)**.** *For a conflict-free guarded table, the following are equivalent.*

1. *Every public parser run starting from some $c_0(w)$, $w \in T^*$, is finite.*
2. *The table is phase-terminating in the sense of Definition 21.*
3. *The table has a termination certificate in the sense of Definition 24.*

When these conditions hold, the canonical exact rank ρ_{ex} of Definition 27 is a termination certificate.

Proof. (1) $\Rightarrow$ (2)). Assume every public parser run is finite, and suppose for contradiction that there are $t \in T'$ and an infinite chain

$$s_0 \Rightarrow_t s_1 \Rightarrow_t s_2 \Rightarrow_t \cdots$$

with $\mathsf{Phase}(s_0, t)$. By definition of Phase, there are $u \in T'^*$ and $w \in T^*$ such that

$$c_0(w) \vdash^*_{\mathbf{LA}} (s_0, tu).$$

By Lemma 20, each edge $s_i \Rightarrow_t s_{i+1}$ is exactly a guarded Reduce step

$$(s_i, tu) \vdash_{\mathbf{LA}} (s_{i+1}, tu).$$

Appending these steps to the finite prefix from $c_0(w)$ yields an infinite public parser run, a contradiction. Hence the table is phase-terminating.

(2) $\Rightarrow$ (3)). Assume phase-termination. Let $c = (s, u)$ be reachable. If $u = tu'$, then $\mathsf{Phase}(s, t)$. By Lemma 22 and Lemma 23, the maximum defining $H_t(s)$ exists; the same lemma also gives strict decrease along every exact edge $s \Rightarrow_t s'$.

We prove that ρ_{ex} is a termination certificate. If the recursive step from c to c' is a Shift and $\text{input}(c) = tu'$, then $\text{input}(c') = u'$, so

$$|\text{input}(c')| < |\text{input}(c)|.$$

Therefore $\mu(c') <_{\text{lex}} \mu(c)$, independently of the second component.

If the recursive step is a Reduce, write

$$c = (s, tu'), \qquad c' = (s', tu').$$

By Lemma 20, $s \Rightarrow_t s'$. The height decrease from Lemma 23 gives

$$\rho_{\text{ex}}(c') = H_t(s') < H_t(s) = \rho_{\text{ex}}(c).$$

The input length is unchanged by a Reduce step, so again

$$\mu(c') <_{\text{lex}} \mu(c).$$

Accept and Error configurations make no recursive call. Thus ρ_{ex} is a termination certificate.

(3) $\Rightarrow$ (1)). Assume a termination certificate ρ exists. Along every reachable recursive parser step, the measure

$$\mu(c) = (|\text{input}(c)|, \rho(c))$$

strictly decreases in the lexicographic order on $\mathbb{N} \times \mathbb{N}$. This order is well-founded, so no infinite sequence of reachable recursive calls can start from any $c_0(w)$. Hence every public parser run is finite. □

Corollary 29 (Fuel-free recursive parser). *If the equivalent conditions of Theorem 28 hold, the public parser can be exposed without a fuel parameter:*

$$\text{run\_parser} : T^* \to \text{option ParseTree}.$$

Proof. Use the certificate rank to order recursive calls by

$$c' \prec c \quad \Longleftrightarrow \quad \mu(c') <_{\text{lex}} \mu(c).$$

The lexicographic order on $\mathbb{N}\times\mathbb{N}$ is well-founded, and the certificate supplies the required decrease for every reachable Shift or Reduce call. Thus the internal runner is defined on reachable configurations by well-founded recursion on $\prec$, equivalently by carrying a reachability proof internally. The initial configuration $c_0(w)$ is reachable by the empty run, so the public wrapper supplies the accessibility proof and exposes no fuel argument. □

The criterion above is exact: a table is rejected only when there is an actual infinite reduce-only phase reachable from some initial input. A finite control-only graph may still be useful as a sufficient quick check, but it is not used as the complete criterion because such a graph can contain cycles that no single parser stack can realise.

References

[1] F. DeRemer and T. J. Pennello. Efficient computation of LALR(1) look-ahead sets. *ACM Transactions on Programming Languages and Systems*, 4(4):615–649, 1982.

[2] R. E. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

[3] J. H. Gallier. A survey of LR-parsing methods: The graph method for computing fixed points and LALR(1) lookahead sets. CIS 511 lecture notes, University of Pennsylvania, 2008.